

MÓDULO 11: MASTERCLASS

DOMAIN-DRIVE DESIGN DESDE CERO A CÓDIGO

BIG school

GENERAMOS
NEGOCIO

BIGSEO



DDD

Desmitificando el “Diseño Orientado a Dominio”

BIG school



Domain Drive Design

BIG school

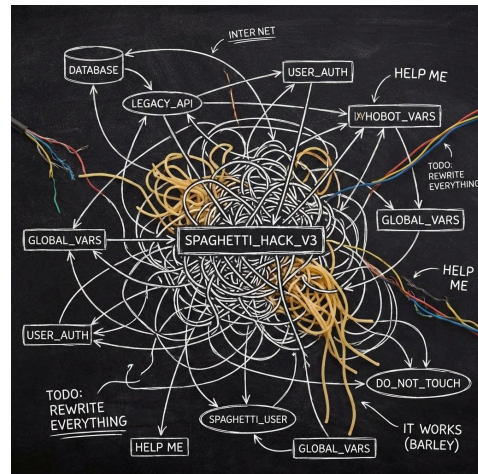
BLOQUE 1:

INTRODUCCIÓN Y ESTRATEGIA

Domain-Driven Design: De la Teoría al Código.

La gran bola de m... lodo (Big Ball of Mud)

- **Síntoma:** Cambias una línea en "Facturación" y rompes "Envíos".
- **Causa:** Complejidad técnica desalineada con el negocio.
- **Resultado:** Miedo a tocar el código legacy.

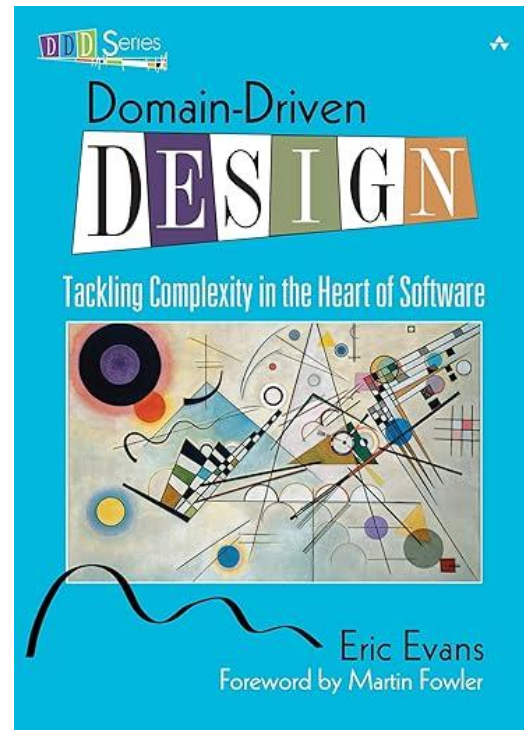


Cambio de paradigma

"La tecnología es lo de menos. Lo difícil es modelar el negocio." — Eric Evan

Antes: Design by Data (Tablas, FKs, EJBs).

Ahora: Design by Domain (Lenguaje, Reglas, Comportamiento).



El Anti-Patrón

✗ Empezar dibujando tablas (ID, Nombre, Fecha).

✗ "Inventariar tornillos antes de tener los planos de la casa."

⚠ Resultado: **Modelos Anémicos** (Bolsas de datos con Getters/Setters).

Lenguaje Ubicuo

Si el experto dice "Matricular",
el código dice `.matricular()`.
NO `.insertRow()`.



Contextos delimitados (Bounded

C)



La polisemia es el enemigo

Contexto Ventas: "Pedido" = Dinero + Artículos.

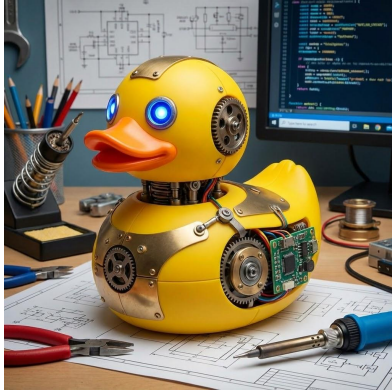
Contexto Almacén: "Pedido" = Caja (Peso + Dimensiones).

Contexto Envíos: "Pedido" = Ruta + Tracking ID.

No busques un modelo único para toda la empresa.

Divide y vencerás.

Interrogando al Negocio (Rubber Ducking 2.0)



Uso de la IA:

- Analizar requisitos crudos y ambiguos.
- Detectar conflictos de lenguaje (Polisemia).
- Simular escenarios antes de programar.

Desata el nudo semántico antes de escribir código

El puente: BDD & Gherkin

Especificación ejecutable sin hablar de bases de datos.

```
Given el curso tiene 20 plazas  
When "Carlos" intenta matricularse  
Then la matrícula es aceptada  
And el cupo baja a 19
```



Domain
Drive
Design

BIG school

BLOQUE 2:

DISEÑO TÁCTICO

Value Objects

Concepto:

- ¿`email: string`? **✗** (Acepta basura).
- ¿`cupo: int`? **✗** (Acepta -5).

Características de un V.O.:

1. Inmutabilidad (`readonly`).
2. Validación en la creación (Constructor/Factory).
3. Identidad basada en valor (5€ es igual a 5€).

Limpiando *Legacy* con IA

Estrategia:

- Input: Clase anémica con primitivos.
- Prompt: "*Refactoriza a DDD estricto, encapsula y valida.*"



Advertencia: Tú eres el arquitecto.

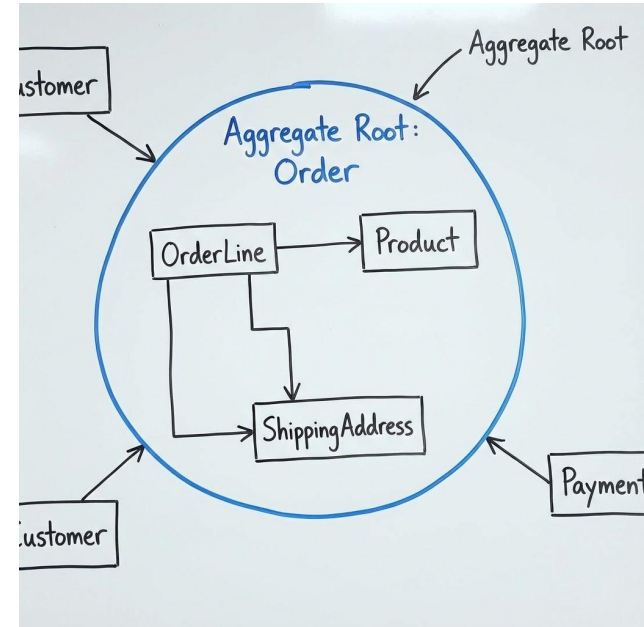
Revisa los *accessors* y la lógica generada

Entidades y Agregados

Proteger las invariantes

Reglas:

- **Course (Root):** El jefe. Nadie toca los internos sin pasar por él.
- **Invariantes:** Reglas de consistencia inmediata (ej. "No superar aforo").
- **Encapsulamiento:** `private _students`. Solo exponer comportamiento (`enroll()`).



Application Services

Capa de Aplicación (Use Cases)

Rol: El "Bibliotecario" / Director de Orquesta.

Qué HACE:

- Orquestrar la transacción.
- Buscar en Repositorio -> Ejecutar Dominio -> Guardar.

Qué NO hace:

- Lógica de negocio (eso es del Dominio).
- Lógica HTTP (eso es del Controlador).



Domain
Drive
Design

BIG school

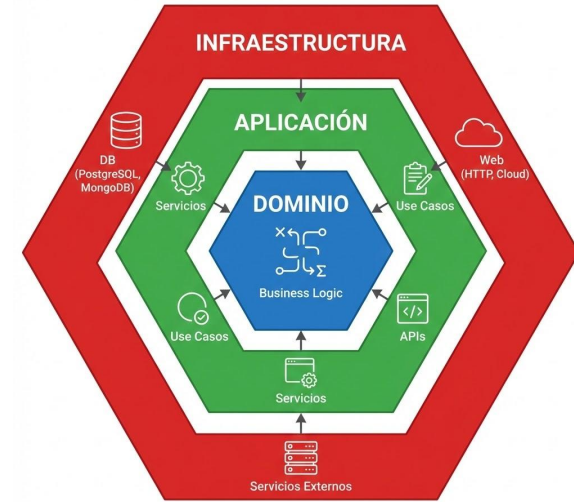
BLOQUE 3:

**ARQUITECTURA E
INFRAESTRUCTURA**

Arquitectura Hexagonal

El puerto seguro

- Dependencias hacia adentro. El Dominio no conoce a Mongo.
- **Puertos:** Interfaces (`ICourseRepository`).
- **Adaptadores:** Implementaciones (`MongoCourseRepository`).



Persistencia

Objetos ricos (Agregados)
vs. Documentos planos (JSON).

Solución: Mappers explícitos.

Diferencia Clave:

- `Course.create()`: Valida reglas estrictas (Nuevos cursos).
- `Mapper.toDomain()`: Reconstituye hechos pasados (Cargar de DB).

Testing como documentación

Tests rápidos unitarios porque no tocan la Base de Datos.

```
expect(() => course.enroll(...)).toThrow("No hay cupo")
```

Conclusiones

Diseña para el Dominio, no para la DB.

- Habla el idioma del experto.
- Aísla la complejidad en el núcleo.
- Usa la IA para acelerar el descubrimiento y el refactor.

Tu Misión para Mañana

No reescribas el monolito. Busca una
pequeña victoria.

Encuentra un `string phoneNumber` y
conviértelo en `Class PhoneNumber`.

BIG school